Implementatie- & Deploymentplan

UNASAT Campus Support Chatbot

Beroepsproduct | 2024-2025

UNASAT Business School

Opleiding: Bachelor ICT

Vak: AI Solutions

Groepsleden:

Bindya Kewalapat (SE/1123/034) | Dharandjai Patan (SE/1123/056) | Masjermi Waakzaam (SE/1123/101)

Melvin Ritfeld (SE/1123/075) | Wasil Nanhekhan (SE/1123/052)

Datum: 7 maart 2026

1. Deploymentstappen

Dit stappenplan is ontworpen om praktisch en direct uitvoerbaar te zijn. Elke stap kan probleemloos worden uitgevoerd door een IT-medewerker, ongeacht eventuele voorkennis van de codebase.

Stap 1: Vereisten controleren

Zorg ervoor dat de volgende componenten en gegevens beschikbaar zijn voordat de deployment start:

- Docker Desktop geinstalleerd (minimaal versie 24.0)
- Git geinstalleerd op de hostmachine
- Een actieve internetverbinding voor het ophalen van Docker images en communicatie met de Groq API
- Een gratis Groq API key, aan te maken via console.groq.com

Stap 2: Repository klonen

Kloon de projectbestanden naar de lokale machine met de volgende commando's:

```
git clone https://github.com/MelvinRitfeld/production-chatbot
cd production-chatbot
```

Stap 3: Geautomatiseerde setup uitvoeren

Start de geautomatiseerde installatie op basis van je besturingssysteem:

Windows: powershell -ExecutionPolicy Bypass -File setup.ps1

Linux / Mac: chmod +x setup.sh && ./setup.sh

Het script voert de volgende acties volledig geautomatiseerd uit:

- Controleert of Docker correct is geinstalleerd en actief is
- Kopieert het backend/.env.example bestand naar een werkbaar backend/.env bestand
- Vraagt de gebruiker om de GROQ_API_KEY en wacht op invoer
- Valideert of de ingevoerde API key geldig is
- Start de applicatie op via docker compose up --build -d
- Toont een overzicht van alle toegangs-URLs zodra het systeem online is

Stap 4: Verificatie

De opstarttijd bedraagt 5 tot 15 minuten bij de eerste run, afhankelijk van de downloadsnelheid. Controleer de beschikbaarheid via de volgende URLs:

Onderdeel	URL
Chatbot interface	http://localhost:3000
Admin dashboard	http://localhost:3000/admin
Backend API docs	http://localhost:8000/docs
Health check	http://localhost:8000/health

Stap 5: Productie deployment via Railway (live)

De applicatie is tevens live gedeployed op Railway en publiek bereikbaar zonder lokale installatie:

Onderdeel	URL
Chatbot interface	https://renewed-recreation-production-a5fa.up.railway.app
Admin dashboard	https://renewed-recreation-production-a5fa.up.railway.app/admin
Backend API	https://production-chatbot-production-6f5a.up.railway.app

Railway herdeployt automatisch bij elke push naar de main branch via GitHub. Geen handmatige stappen vereist.

2. Omgevingen

Het systeem ondersteunt drie omgevingen: development, lokale productie en cloud productie.

Omgeving	Doel	Configuratie	Database
Development	Lokaal ontwikkelen en testen	backend/.env met lokale waarden	Lokale PostgreSQL container
Productie (lokaal)	Live gebruik via Docker Compose	backend/.env met productie API key	PostgreSQL met persistent volume
Productie (Railway)	Live cloud deployment - actief	Omgevingsvariabelen via Railway dashboard	Railway Managed PostgreSQL

💡 De Railway productieomgeving is actief op: renewed-recreation-production-a5fa.up.railway.app

3. Configuratiebeheer

Veiligheid staat voorop: alle configuratie verloopt via omgevingsvariabelen. Secrets worden nooit hardcoded in de codebase of opgeslagen in het Git-repository. Het sjabloon backend/.env.example bevat enkel placeholder-waarden en mag gecommit worden; het echte .env bestand is afgeschermd via .gitignore.

Variabele	Locatie	Beschrijving	Opgeslagen in Git?
GROQ_API_KEY	backend/.env / Railway dashboard	API sleutel voor de Groq LLM	Nee
DATABASE_URL	backend/.env / Railway dashboard	PostgreSQL connectiestring	Nee
NEXT_PUBLIC_API_BASE_URL	Railway frontend variabelen	Base-URL van de backend voor de frontend	Nee (Railway)
POSTGRES_DB / USER / PASSWORD	docker-compose.yml	Database inloggegevens (lokaal)	Ja (standaard dev-waarden)

4. Versiebeheerstrategie

Voor versiebeheer wordt gebruikgemaakt van Git in combinatie met GitHub. We hanteren een strikte branching- en commitstrategie om de stabiliteit te waarborgen. Voorafgaand aan elke push naar de repository wordt de code lokaal getest met het commando docker compose up --build.

Branches:

- main: De stabiele productieversie. Deze branch is direct gekoppeld aan de Railway productie-deployment en triggert automatisch een redeploy bij elke push.
- feature/*: Voor de ontwikkeling van nieuwe functionaliteiten. Deze integreren we via een pull request.
- fix/*: Gereserveerd voor bugfixes, welke na een code review direct gemerged worden naar main.

Commitconventies:

- feat: Toevoeging van nieuwe functionaliteit
- fix: Oplossen van een bug
- docs: Wijzigingen in de documentatie
- chore: Aanpassingen in configuratie of het build-proces

5. Updateprocedure

Afhankelijk van het type update, gelden er specifieke procedures.

Applicatie Update (Code wijzigingen)

Lokaal:

- Pull de laatste versie: git pull origin main
- Herbouw en herstart: docker compose up --build -d
- Controleer de API-status: curl http://localhost:8000/health
- Verifieer de werking via het admin dashboard op http://localhost:3000/admin

Railway (productie):

- Push naar main: git push origin main
- Railway detecteert de push en herdeployt automatisch binnen 2-3 minuten
- Verifieer via het Railway dashboard of de deployment succesvol is

FAQ Kennisbank Update

- Open en bewerk backend/app/data/answer_bank.py
- Voeg nieuwe entries toe volgens het bestaande datamodel
- Lokaal: docker compose up --build -d backend
- Railway: git push origin main (automatische redeploy)
- Test de nieuwe vragen via de chatinterface

API Key Rotatie

- Genereer een nieuwe key in het Groq console
- Lokaal: werk de variabele GROQ_API_KEY bij in backend/.env en herstart: docker compose restart backend
- Railway: pas de GROQ_API_KEY aan in het Railway dashboard van de backend service en herstart de service

6. Rollbackstrategie

Indien een update kritieke problemen veroorzaakt, kan het systeem snel hersteld worden via twee methoden:

Methode 1: Git Revert (Aanbevolen)

- Bekijk de recente commits: git log --oneline -10
- Draai terug naar de laatste werkende status: git revert HEAD
- Herbouw de containers: docker compose up --build -d
- Railway: de revert-commit triggert automatisch een redeploy naar de vorige versie

Methode 2: Volledige Reset (Noodgreep)

⚠️ Het commando docker compose down -v wist alle volumes, inclusief de opgeslagen logs en gesprekken.

- Verwijder de huidige containers inclusief volumes: docker compose down -v
- Check out een specifieke, stabiele commit: git checkout <stabiele-commit-hash>
- Start de applicatie schoon op: docker compose up --build -d

7. Beheer & Verantwoordelijkheden

Het technische en inhoudelijke beheer is als volgt verdeeld:

Taak	Verantwoordelijke	Frequentie
FAQ kennisbank uitbreiden	Contentbeheerder / UNASAT medewerker	Maandelijks
Groq API key bewaken	Technisch beheerder	Maandelijks
Logs en metrics reviewen	Technisch beheerder	Wekelijks
Docker / Railway updates	Technisch beheerder	Per release
Ongematchte vragen analyseren	Contentbeheerder	Wekelijks
Feedbackratings reviewen	Contentbeheerder	Wekelijks
Security patches uitvoeren	Technisch beheerder	Direct na melding

💡 Het admin dashboard op /admin biedt direct, zonder benodigde technische kennis, inzicht in belangrijke metrics zoals latentie, FAQ match rate en recente logs.

8. Kosteninschatting

De architectuur is specifiek ontworpen om kosten te minimaliseren door zoveel mogelijk gebruik te maken van de FAQ-afvang. Antwoorden vanuit de FAQ genereren namelijk geen API-kosten. Bij een match rate van 60% worden de API-kosten met 60% gereduceerd vergeleken met een puur LLM-gedreven oplossing.

Huidige opzet (Railway productie — actief)

Component	Maandelijkse Kosten	Limieten & Opmerkingen
Groq API	€0	Maximaal ~14.400 tokens/minuut, tot 30 verzoeken per minuut
Railway Frontend	~€2	Berekend op basis van CPU/geheugengebruik
Railway Backend	~€2	Lichte service, minimaal resourcegebruik
Railway PostgreSQL	~€1	Managed database met persistent volume
GitHub Repo	€0	Gratis (public repository)
Totaal (Huidig)	~€5/maand	Valt binnen Railway gratis krediet van $5/maand

Eerder overwogen alternatief (VPS)

Aanvankelijk was de planning om de applicatie op een VPS (DigitalOcean of Hetzner, ~€5-€20/maand) te draaien via Docker Compose. Er is uiteindelijk gekozen voor Railway vanwege automatische GitHub-integratie, managed PostgreSQL en ingebouwde HTTPS zonder extra configuratie.

Geschatte kosten bij opschaling

Scenario	Component	Geschatte Kosten
~50 studenten/dag	Groq gratis tier	€0 (valt ruim binnen de gratis limieten)
~500 studenten/dag	Groq betaald plan	~€15 tot €30 per maand
Grotere cloud hosting	Railway Pro of VPS	€10 tot €20 per maand
Managed Database (groot)	Railway of Supabase betaald	€5 tot €10 per maand
Totaal (groot gebruik)	Volledige stack	~€20 tot €50 per maand